

Quick Start

Simple steps for producers and consumers

This short guide tells you **what to do** to send and receive messages through the broker. It uses plain words and small .NET examples.

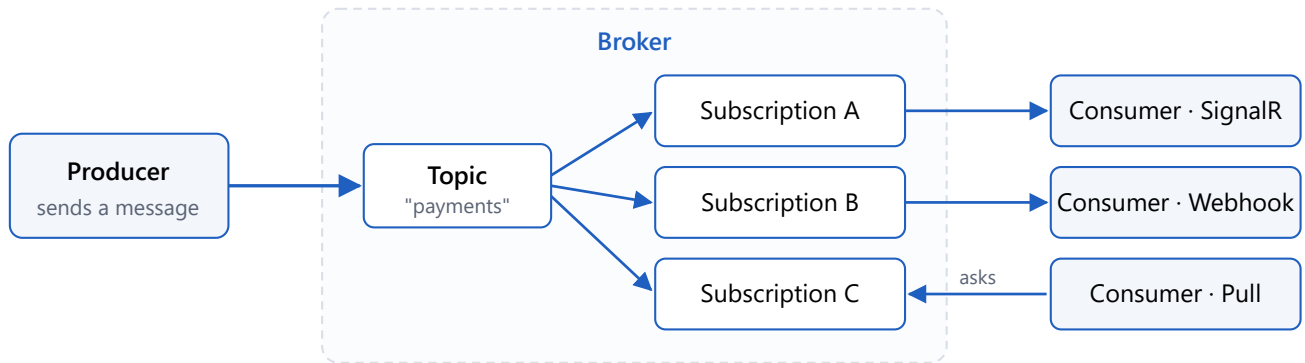
Inside:

1. The idea in one page
2. Before you start: what to ask the admin for
3. Producer: how to send a message
4. Producer from a stored procedure
5. Producer from an ActiveBatch job
6. Rules for every consumer
7. Consumer with SignalR
8. Consumer with Webhook
9. Consumer with Long polling (Pull)
10. Which one should I pick?
11. When something goes wrong

For every detail and option, see the full User Guide (MessageBroker-User-Guide.pdf).

1. The idea in one page

A **producer** is an app that sends messages. A **consumer** is an app that receives them. They never talk to each other directly. The **broker** sits in the middle: it stores every message safely and hands it to each consumer that wants it.



The producer sends one message to a **topic**. The broker makes one copy for every **subscription** on that topic. Each consumer owns a subscription and gets its own copy in one of three ways: the broker **pushes** it over SignalR, the broker **calls** the consumer's web address (Webhook), or the consumer **asks** for it (Pull with long polling).

Words you will see

Word	Meaning
Topic	A named channel for one kind of news, for example <code>payments</code> . Producers send to a topic.
Message	What the producer sends: a type name (like <code>PaymentProcessed.v1</code>) and a JSON payload with the data.
Subscription	A consumer's "mailbox" on a topic. Every subscription gets its own copy of every message.
Delivery	One copy of a message, on its way to one subscription.
ACK	"Done, thank you." The consumer tells the broker the delivery was handled. It will not come again.
NACK	"It failed, try again later." The broker waits a little and delivers it again.
Lease (lock)	The time a consumer has to finish a delivery. If it does not answer in time, the broker gives the delivery out again.
Dead-letter queue (DLQ)	Where a message goes when it failed too many times or the consumer said "this can never work". Nothing is lost; the admin can look at it and send it again.
API key	Your app's password for the broker. Every call carries it.

Good to know: the broker may sometimes deliver the same message **twice** (for example after a network error). This is normal. Every consumer must be able to handle a repeat safely (see chapter 6).

2. Before you start: ask the admin

You do not create topics, keys or subscriptions yourself. The broker admin does that for you. Tell the admin what your app is, and you will get back what you need:

You are...	Tell the admin	You get back
Producer	Your app's name and which topic you want to send to.	An API key and the topic name .
Producer · stored procedure	Your app's name and the topic. Your DBA also runs the outbox script once (chapter 4).	An API key for the relay that sends the events, and the topic name .
Producer · ActiveBatch job	The job's name and the topic.	An API key (keep it in ActiveBatch's secure store) and the topic name .
Consumer · SignalR	Your app's name, the topic, and "SignalR mode".	An API key and a subscription ID (a GUID).
Consumer · Webhook	Your app's name, the topic, "Webhook mode", and your HTTPS address , for example <code>https://billing.internal/webhooks/payments</code> .	An API key , a subscription ID and a webhook secret (used to check that calls really come from the broker).
Consumer · Pull	Your app's name, the topic, and "Pull mode".	An API key and a subscription ID .

You also need the broker's address, for example `https://broker.internal/`.

Keep secrets secret. The API key and the webhook secret are shown **only once**. Put them in your secret store (Key Vault, environment settings). Never put them in source code or Git, and never write them to logs.

Add the client package (all apps)

.NET apps use the `MessageBroker.Contracts` package. It has the ready-made client, the SignalR listener and the webhook signature checker. Add it to your project, then put the broker settings in configuration:

```
// appsettings.json (the key itself comes from your secret store)
"Broker": { "Url": "https://broker.internal/", "ApiKey": "<from secret store>" }
```

3. Producer: how to send a message

Sending is one HTTP call. The broker answers only after the message is safely stored.

Step 1 Register the broker client

In `Program.cs`, register `BrokerClient` with the broker address and your API key.

```
builder.Services.AddBrokerClient(o =>
{
    o.BaseAddress = new Uri(builder.Configuration["Broker:Url"]);
    o.ApiKey      = builder.Configuration["Broker:ApiKey"];
});
```

Step 2 Build the message

Give it a **type name with a version** (`PaymentProcessed.v1`) and put your data in the **payload**. Keep it small (under 256 KB) and do not put passwords or secrets in it. The **correlation ID** is optional: use your order or request ID so the whole flow can be traced later.

```
var request = new PublishRequest
{
    MessageType = "PaymentProcessed.v1",
    CorrelationId = payment.OrderId,
    Payload      = JsonSerializer.SerializeToElement(
        new { payment.PaymentId, payment.Amount })
};
```

Step 3 Send it, with an idempotency key

The **idempotency key** is a name for this business event, for example `payment-PAY-123-processed`. If you send the same key twice, the broker keeps only the first one. So you can safely send again after an error without making a duplicate.

```
var result = await broker.PublishAsync("payments", request,
    idempotencyKey: $"payment-{payment.PaymentId}-processed", ct: ct);

// result.MessageId    → the new message's ID (log it for tracing)
// result.IsDuplicate  → true if this key was already sent before (still OK)
```

Step 4 Look at the answer

Answer	What it means → what you do
201 Created	Stored. You are done.
200 OK	Already sent earlier with this key. Treat it as success.
400 / 413	The message is wrong or too big. Fix it. Do not just send it again.
401 / 403 / 404	Wrong key, no permission, or no such topic. Check your settings or ask the admin.
503 , timeout, network error	The broker is busy or down. Try again later with the same key (wait 1, 2, 4, 8... seconds).

With the .NET client, errors arrive as `BrokerApiException`. Its `IsTransient` property is true when it is safe to try again.

Do

- ✓ Always send an idempotency key.
- ✓ Retry only on 503, timeouts and network errors.
- ✓ Put a version in the type name (`.v1`, `.v2`).
- ✓ Save the message in your own database first (an "outbox" table) if losing it would be bad.

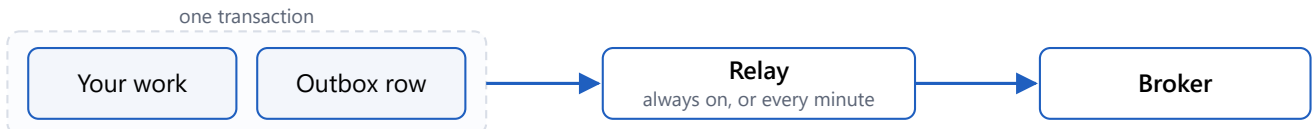
Don't

- ✗ Don't retry 4xx errors unchanged.
- ✗ Don't put secrets or large files in the payload.
- ✗ Don't log your API key.
- ✗ Don't expect consumers to get messages in exact order.

4. Producer from a stored procedure

Use this when the event happens inside the database: a stored procedure changes data, and other apps must hear about it ("order shipped", "price changed").

Can a stored procedure call the broker directly? No. SQL Server 2022 cannot call a web address by itself, and it would not be safe anyway: if the transaction rolls back after the call, the event was already sent for something that never happened. Instead, the procedure **writes the event into an outbox table**, in the same transaction as its work. A small program, the **relay**, then sends the outbox rows to the broker.



Step 1 One-time setup

Your DBA creates the outbox table and its procedures in your database, from `samples/Samples.Shared/sample.sql` (Outbox, `usp_Outbox_Enqueue`, `usp_Outbox_GetPending`, `usp_Outbox_MarkSent`, `usp_Outbox_MarkFailed`). Ask the admin for an API key and the topic name for the relay (chapter 2).

Step 2 In your procedure, queue the event inside your transaction

Call `usp_Outbox_Enqueue` between `BEGIN TRAN` and `COMMIT`. If anything fails, the rollback removes your change **and** the event together, so they always match.

```
CREATE OR ALTER PROCEDURE dbo.usp_Order_Ship @OrderId nvarchar(50)
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;
    BEGIN TRAN;
        UPDATE dbo.Orders SET Status = 'Shipped' WHERE OrderId = @OrderId;    -- your work

        DECLARE @Payload nvarchar(max) =
            (SELECT @OrderId AS orderId, SYSUTCDATETIME() AS shippedAt
             FOR JSON PATH, WITHOUT_ARRAY_WRAPPER);

        EXEC sample.usp_Outbox_Enqueue                                     -- the event
            @TopicName = N'orders', @MessageType = N'OrderShipped.v1',
            @Payload = @Payload, @CorrelationId = @OrderId;
    COMMIT;
END
```

Step 3 Run a relay

Pick one: the always-running **SamplePublisher** sends new rows within about a second. Or a scheduler (ActiveBatch, SQL Agent, Task Scheduler) runs `SamplePublisher.exe relay-once` every minute: it sends everything waiting, then stops (chapter 5, Way B). Each row is sent with the key `outbox-<OutboxId>`, so a row is never published twice, even if a run is cut off.

Step 4 Keep an eye on the outbox

```
-- Waiting rows. Growing, or very old? The relay is stopped or the broker is down.
SELECT COUNT(*) AS Waiting, MIN(CreatedAt) AS OldestWaiting
FROM sample.Outbox WHERE SentAt IS NULL AND RejectedAt IS NULL;

-- Rows the broker refused (bad event): fix the cause and queue the event again.
SELECT OutboxId, MessageType, LastError FROM sample.Outbox WHERE RejectedAt IS NOT NULL;
```

Try it: EXEC sample.usp_Payment_Record 'PAY-SQL-1', 250, 'INR', 'payments' records a payment and queues its PaymentProcessed.v1 event. Run relay-once, and the sample subscribers receive it.

5. Producer from an ActiveBatch job

Use this when a scheduled job must publish its own event ("nightly import finished"), or send the events your stored procedures queued. The job runs a command and looks at its **exit code**.

Exit code	Meaning	Set the job to...
0	Done (or already published with this key).	Succeed.
1	Refused: wrong key, topic or event.	Fail and alert someone. Don't retry.
2	Broker unreachable. Nothing was lost.	Fail, restart later (e.g. 3 times, 1 min apart).

Way A: the job publishes its own event

Step 1 Keep the API key in ActiveBatch's secure store

Use a secure (hidden) variable or a credential object, and give it to the job as the environment variable `BROKER_API_KEY`. Never put the key on the command line: it would show in the job history.

Step 2 Add a step that runs the script

Copy `samples/ActiveBatch/Publish-BrokerEvent.ps1` to the job's machine. As the last step of the job, after the real work, add a **PowerShell script** step:

```
$payload = @{ title = 'Nightly import'; text = '1,250 rows loaded'; level = 'Success' } |  
    ConvertTo-Json -Compress  
.\Publish-BrokerEvent.ps1 -BrokerUrl https://broker.internal/ -Topic notifications `   
    -MessageType UserNotification -Payload $payload `   
    -IdempotencyKey "nightly-import-$(Get-Date -Format yyyy-MM-dd)"  
exit $LASTEXITCODE
```

The script retries a busy broker by itself (3 tries, with the same key) before it gives up with exit code 2. If the step can only run a command line, save the JSON in a file and pass `-PayloadFile event.json` instead: quotes inside JSON do not survive a Windows command line.

Step 3 Choose the idempotency key well

Name the **business event**, not the run: `nightly-import-<date>`, `invoice-batch-<batchId>`. Then a rerun of the same job for the same day does not publish twice. `Example-NightlyJob.ps1` shows a whole job built this way.

Way B: the job sends what stored procedures queued

Step 1 Schedule the relay every minute

Create a job that runs `SamplePublisher.exe relay-once` every minute, with "do not start while the previous run is still running". It sends all waiting outbox rows (chapter 4), prints for example `relay-once: 12 sent, 0 rejected`, and exits with the codes above. Here exit code 2 also covers a wrong key or topic: the rows simply wait, and the printed warning says why.

Menu names differ between ActiveBatch versions: `samples/ActiveBatch/README.md` has the full job setup. The same commands work in SQL Agent and Windows Task Scheduler.

6. Rules for every consumer

Whatever way you choose to receive (SignalR, Webhook or Pull), follow these five rules.

1. **Be ready for repeats.** The same message can arrive twice. Save the `messageId` of every message you have handled (in your database, in the same transaction as your work). If you see it again, just say "done" (ACK) and do nothing else.
2. **Say "done" only after your work is saved.** If you ACK first and then crash, the message is lost for you.
3. **Answer in time.** Each delivery has a deadline, `lockedUntil`. After it, the broker thinks you failed and sends the delivery again. For long work, ask for more time (renew).
4. **NACK to try again, dead-letter to give up.** A temporary problem (database busy, other service down)? NACK, and the broker tries again later. A message that can never work (bad data, unknown type)? NACK with *dead-letter*, and it goes straight to the DLQ.
5. **Never log the payload or keys.** Log `messageId`, `correlationId` and `deliveryId` instead.

What a delivery looks like

All three ways give you the same JSON:

```
{
  "deliveryId": 81234,                                ← used to ACK / NACK / renew
  "lockToken": "2c9a0f4e-5d71-4b0e-9b77-0c4b9a2f1e11", ← proves this delivery is yours right now
  "lockedUntil": "2026-10-08T12:31:05Z",               ← answer before this time (UTC)
  "attempt": 1,                                       ← 1 the first time, 2 the second time, ...
  "message": {
    "messageId": "01a11b48-b86f-784f-a72d-8e1bc7526202", ← check this for repeats
    "messageType": "PaymentProcessed.v1",
    "correlationId": "ORDER-48213",
    "createdAt": "2026-10-08T12:30:04Z",
    "payload": { "paymentId": "PAY-12345", "amount": 1500.00 }
  }
}
```

How many times does the broker try?

By default, 4 times, waiting about 30 s, then 60 s, then 120 s between tries. After the last failed try, the message goes to the DLQ. If your app may be down for longer, ask the admin to give your subscription more tries.

7. Consumer with SignalR

Choose SignalR when your app cannot offer a web address the broker can call (desktop app, behind a firewall, Blazor Server), or you want messages **instantly**. Your app opens one connection to the broker and keeps it open; the broker pushes each message down that connection.

Step 1 Get your details from the admin

You need the broker address, your **API key** and your **subscription ID**.

Step 2 Write the handler: what to do with one message

Return `DeliveryResult.Ack` when the work is saved. Return `DeliveryResult.Fail(...)` to try again later, or add `deadLetter: true` to give up on it. If your code throws an exception, that counts as "try again".

```
async Task<DeliveryResult> HandleAsync(Delivery delivery, CancellationToken ct)
{
    var type = delivery.Message.MessageType;
    if (type != "PaymentProcessed.v1")
        return DeliveryResult.Fail("UnknownType", type, deadLetter: true); // give up

    await ProcessOnceAsync(delivery, ct); // your work (skips repeats)
    return DeliveryResult.Ack;           // done
}
```

Step 3 Create the listener and start it

`SignalRDeliveryListener` connects, signs in with your key, listens on your subscription, and sends ACK or NACK for you based on your handler's answer. Run it in a `BackgroundService` so it lives as long as your app.

```
public sealed class PaymentListener(IConfiguration config) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stop)
    {
        var brokerUrl = new Uri(config["Broker:Url"]!);
        var subscriptionId = Guid.Parse(config["Broker:SubscriptionId"]!);

        await using var listener =
            new SignalRDeliveryListener(brokerUrl, config["Broker:ApiKey"]!, HandleAsync);

        await listener.StartAsync([subscriptionId], stop);
        await Task.Delay(Timeout.Infinite, stop); // keep listening until the app stops
    }
}

// Program.cs: builder.Services.AddHostedService<PaymentListener>();
```

Step 4 Nothing else: it reconnects by itself

If the network drops or the broker restarts, the listener reconnects on its own. Unanswered deliveries come again after their deadline, which is safe because you skip repeats. For long work, call `listener.RenewAsync(delivery)` to get more time.

Not .NET? Any SignalR client works: connect to `https://<broker>/hubs/deliveries` with your API key as the access token, call `Subscribe(subscriptionId)`, handle the `Deliver` event, then call `Ack` or `Nack`.

8. Consumer with Webhook

Choose Webhook when your app is a web service that already has an HTTPS address. You do not keep any connection open: **the broker calls you** with an HTTP POST for every message, and your HTTP answer tells it whether it worked.

Step 1 Make an HTTPS endpoint and give its address to the admin

For example `https://billing.internal/webhooks/payments`. It must use HTTPS. The admin creates your subscription with this address and gives you your **webhook secret**.

Step 2 Check that the call really comes from the broker

Every call is signed with your webhook secret. Read the body as plain text **first**, then check it with `WebhookSignature.Verify`. If the check fails, answer **401** and do nothing.

Step 3 Do the work, then answer

You answer	The broker understands
200 (any 2xx except 202)	Done (ACK).
4xx or 5xx	Failed, try again later (NACK).
202 Accepted	"I took it and will report back myself." Use it for long work, or to give up on a message: later call <code>POST /api/v1/deliveries/{id}/ack</code> , or <code>/nack</code> with <code>"deadLetter": true</code> . The ID and lock token are in the <code>X-Broker-Delivery-Id</code> and <code>X-Broker-Lock-Token</code> headers.
No answer within 30 s	Failed (timeout), try again later.

The whole endpoint:

```
string[] secrets = [builder.Configuration["Broker:WebhookSecret"]!];

app.MapPost("/webhooks/payments", async (HttpRequest request, PaymentHandler handler) =>
{
    using var reader = new StreamReader(request.Body);
    var body = await reader.ReadToEndAsync();           // 1. read the raw text first

    var signature = request.Headers[WebhookHeaders.Signature];
    var timestamp = request.Headers[WebhookHeaders.Timestamp];
    if (!WebhookSignature.Verify(signature, timestamp, body, DateTimeOffset.UtcNow, secrets))
        return Results.Unauthorized();                 // 2. not from the broker

    var delivery = JsonSerializer.Deserialize<Delivery>(body, JsonSerializerOptions.Web)!;
    try
    {
        await handler.ProcessOnceAsync(delivery);       // 3. your work (skips repeats)
        return Results.Ok();                           // → done (ACK)
    }
    catch (Exception)
    {
        return Results.StatusCode(503);                 // → try again later
    }
});
```

Good to know: after 5 failures in a row, the broker stops calling you for 60 seconds; messages wait safely meanwhile. When the admin changes your secret, the old and new ones both work for 24 hours: put both in `secrets` during that time.

9. Consumer with Long polling (Pull)

Choose Pull when you want to decide yourself **when** and **how fast** to take messages: background workers, batch jobs, or work you want to spread over several copies of your app. Your app **asks** the broker for messages in a loop.

What is long polling? Your app asks "any messages?" and the broker **waits** (up to 20 seconds here) before answering. As soon as a message arrives, the answer comes back at once. If nothing arrives, you get an empty list and simply ask again. So your app reacts instantly, but does not flood the broker with questions.

Step 1 Get your details from the admin

You need the broker address, your **API key** and your **subscription ID**. Register `BrokerClient` as in chapter 3, step 1.

Step 2 Ask for messages in a loop

Call `ReceiveAsync` with how many messages you want at most (1–32) and how long to wait (0–30 s; use 10–30).

Step 3 Handle each message and answer for each one

Every message in the batch has its own deadline. Call `AckAsync` when the work is saved, or `NackAsync` to try again later (add `deadLetter: true` to give up). For long work, call `RenewAsync` to get more time.

The whole worker:

```
public sealed class PaymentWorker(BrokerClient broker, IConfiguration config) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stop)
    {
        var subscriptionId = Guid.Parse(config["Broker:SubscriptionId"]!);
        while (!stop.IsCancellationRequested)
        {
            IReadOnlyList<Delivery> batch;
            try
            {
                // waits up to 20 s, returns at once when a message arrives
                batch = await broker.ReceiveAsync(subscriptionId,
                    maxMessages: 16, waitSeconds: 20, ct: stop);
            }
            catch (BrokerApiException ex) when (ex.IsTransient)
            {
                await Task.Delay(5000, stop);           // broker busy: wait, then ask again
                continue;
            }

            foreach (var delivery in batch)
                await HandleAsync(delivery, stop);
        }
    }

    async Task HandleAsync(Delivery delivery, CancellationToken stop)
    {
        try
        {
            await ProcessOnceAsync(delivery, stop);      // your work (skips repeats)
            await broker.AckAsync(delivery, stop);        // done
        }
        catch (InvalidDataException ex)                  // bad data: give up
        {
            await broker.NackAsync(delivery, "Invalid", ex.Message, deadLetter: true, ct: stop);
        }
        catch (BrokerApiException ex) when (ex.IsLeaseLost)
        {
            // too late: it will come again, and the repeat check handles it
        }
        catch (Exception ex)                             // other problem: try again later
        {
            await broker.NackAsync(delivery, "Error", ex.Message, ct: stop);
        }
    }
}
```

Need more speed? Run more copies of the worker. Each message is given to only one of them at a time.

10. Which one should I pick?

	SignalR	Webhook	Long polling (Pull)
How it works	You keep a connection open; the broker pushes.	The broker calls your HTTPS address.	You ask in a loop; the broker waits until there is something.
Speed	Instant	Instant	Instant (while you are asking)
You need	A long-running app	An HTTPS endpoint the broker can reach	A background loop
Who controls the pace	The broker (up to 8 at once)	The broker (up to 8 at once)	You
Good for	Desktop apps, apps behind a firewall, live screens (Blazor)	Web services and APIs	Workers, batch jobs, scaling out
How you say "done"	Return <code>DeliveryResult.Ack</code>	Answer HTTP 200	Call <code>AckAsync</code>
How you say "try again"	Return <code>DeliveryResult.Fail(...)</code>	Answer 4xx / 5xx	Call <code>NackAsync</code>

Not sure? If you are a web API, pick **Webhook**. If you cannot be called from outside, pick **SignalR**. If you want full control over speed, pick **Pull**.

11. When something goes wrong

You see	What to do
401 Unauthorized	Your API key is missing, wrong or expired. Check your settings; ask the admin for a new key if needed.
403 Forbidden	Your key is fine, but you are not allowed to use this topic or subscription. Ask the admin.
404 Not Found	The topic name or subscription ID is wrong. Check the spelling and the ID.
410 Gone on ACK / NACK	You answered after the deadline. Do nothing: the message comes again and your repeat check handles it. If it happens often, your work is too slow; renew, or ask the admin for a longer lease.
503 Service Unavailable	The broker is busy or restarting. Wait a few seconds and try again.
Webhook: no calls arrive	Is your HTTPS endpoint reachable, and does it answer 2xx? Does the signature check use the right secret? After 5 failures in a row the broker pauses calls for 60 s.
Messages in the DLQ	Look at the error saved with each message (admin dashboard, or ask the admin). Fix the cause, then ask the admin to send the messages again.